前端 Skill 驱动的团队 AI Coding 实践：从个人到整体提效

原创 王僖 阿里技术 Jul 30, 2026, 4:55 AM 浙江 标题已修改 101 listened



这是2026年的第 45 篇文章

（ 本文阅读时间：约 25 分钟 ）

> 前言：当团队里的后端、PD、设计师都在用 AI 写前端代码，我们怎么保证产出是"一个团队的产物"而不是 AI 的随机产物？
> 答案是：**把前端规范做成 Skill，前置到 AI 写第一行代码之前。**

01
背景：AI 能写代码了，但可维护性、规范一致性成了新问题

过去半年，团队几乎全员进入 AI Coding 状态：后端用 AI 写前端模块、PD 用 AI 搭可交互原型。大家跑得很快，但是问题也开逐渐暴露：

- **风格漂移，AI 味浓：** A 同学搭的项目像后台管理模板、B 同学的像官网落地页，布局、配色、交互各走各的，**看不出是一个团队的产物**。
- **技术栈生态不匹配：** AI 偏好 Vite + Next.js + Tailwind + lucide-react 图标库的开源主流搭配，而我们线上是 ice + Fusion / umi + Ant Design（XOps），不加约束就**引入了未经审计的外部依赖**。
- **自创轮子遍地：** `<input>` 配 40 行 CSS 的搜索框、原生 `<button>` 加全套手写样式——项目里到处是 **AI"造"的组件**。
- **信息不对齐：** 大家 AI Coding 不是不愿守规范，是跨端开发时真的不知道规范长什么样；
- **原型活不过评审：** PD 搭的原型在技术栈上和正式开发完全脱节，**评审结束后代码生命周期就终止了**。

技术选型矩阵

👉 一个典型的现象：后端用 AI 全栈跑通一个工具到了验收环节，业务方说"太丑"、设计说"不符合规范"、后端说"前端帮忙调一下"。前端打开代码——Tailwind 散落几十个文件、各种手写 SVG、原生 `<input>` 加一堆css——**功能跑通了，但全都是技术债。**



02
我们尝试过的路径

前端基础课：做了，但不够

我们之前规划过前端课程，也尝试梳理了 系列课程的第一期，但都是前端老生常淡的内容（环境安装、框架、样式、项目运行、调试...），然而现实是，即使后端同学认真学习了，也没法大胆去尝试，因为**真实工程环境里的踩坑经验才是决定代码能不能上线的关键**——这些隐形知识不是一门课能覆盖的。

团队知识库：想做，但是有门槛

团队知识库方向肯定没问题，我们也一直在尝试推进，但对我们当下的场景有两个硬伤：

- **技术债没理清就开始攒规则，知识库本身会变成新的技术债。** 前后端项目技术栈各异、版本参差，存在大量历史坑还未系统梳理过，光是把现状盘清楚短期就做不完——如果边做需求边往里加 rule，会不会出现，知识库成了新的技术债？不好说（摊手）。
- **多人贡献，冲突没人仲裁。** 知识库是团队共享的，不同人对"正确写法"各有各的理解。没有强 owner 和明确的 review 机制，规则之间容易打架，最终没人敢信、没人用。

自建 R2C 平台：尝试后发现不适合我们

去年我们重点关注了 **Aone Super**——一个集团内部的一站式需求研发平台，我们也接入了对方提供的 SDK 并基于此开发了内部的 **Anchor 平台**，做了几轮实测。当时最看重的是它能够覆盖 R2C 全流程，特别是内置**真实环境的在线调试能力**（彼时其他平台还没有这个功能）。

技术选型矩阵

但是几轮测试验证的效果，评估下来对我们团队的现状并不适合——不是平台问题，是**适配度**问题，它更适合有专门资源投入开发运营的场景。具体几点：

- 为了使用平台能力，对项目本身需要做一定配置改造，还需要安装浏览器代理插件，**有一定成本**；
- **长链路稳定性**还在打磨，长任务偶尔中断影响研发节奏；
- **自维护** API Key 模式短期在团队内部跑没问题，长期要实现能力复用和推广，成本不可控（人力维护 + token 消耗）；

我们在内部的 AI 画图工具—— **Andraw 应用**（可以一句话生成高质量的架构图流程图）上也观察到类似现象。「独立平台 + 自维护 API Key」的模式很难走通——我们需要的是**「长在日常工具里、零额外门槛」**的能力。

03
认知转变：从「做平台」到「做 Skill」



接上面的案例，我们做 AI 配图也走过类似的弯路，**先调 prompt → 做独立 App → 最终沉淀为了 Skill**。AI Coding 也是同理：单独做平台对内要持续维护、对外增加门槛，而集团工具能力还在持续进化，**自建平台跟不上就会被淘汰**（One Day、Super Web 现在需要不断和业界领先能力对齐，维护成本很大）。更合理的策略是借 Qoder 这类集团级平台，把精力聚焦在用 Skill、MCP、知识库等来建设符合团队业务特性的周边能力。

那为什么是 Skill 而不是其他？

因为上面尝试的所有路径，都没解决同一个问题：「AI 写代码那一刻」的规范对齐：

- 基础课指望人先学会再动手，但没人看，而且隐性知识传递困难；
- 知识库想把规范、坑点攒齐再给 AI 用，攒不齐不说、攒着攒着还可能变成了债；
- 独立平台在 AI 和代码之间多加了一层，跟日常工作流脱节。

三条路的共同问题在于，**都在 AI 写代码之前加了一个前置动作，而这个动作在实际工作中大概率不会发生**（问自己一句：作为产研，你会愿意脱离自己熟悉的生产力工具，而去使

技术选型矩阵

陌生平台吗？）。Skill 则相反：不用人先上课、不用人先查文档、不用人切到另一个平台，规范直接预加载在你最熟悉的工作流的上下文里，AI 一动手就已经带着约束。

把「老师傅的直觉」写成 AI 能执行的规则

方向定了，真正的难点才开始：

> 💬 ***团队这么多项目、这么多前端烂熟于心但没人沉淀下来过的规矩，怎么塞进一个 AI 能稳定执行的 Skill 里？***

为此，我先把部门里的前端项目摊开做了一轮盘点：

分类	技术栈	特征与诉求	代表平台
经典稳态系	Ice + Fusion + React16 + Formily	阿里系前端老牌技术栈，沉淀久、依赖深，升级成本巨大，核心诉求是**"求稳不翻车"**	故障管理、变更管控等
稳定性标准系	x-ops + ProComponent	已升级技术栈与组件库的团队主力业务项目，规范统一、组件齐备，是当前最标准的一档	AIOps / 风险巡检 / 故障演练 / 云 SPE / 稳定性洞察 / 案例库
通用新建系	React 18 + Ant Design	尚未收口的存量项目和新起项目，技术栈较新但缺乏团队约束，最容易"放飞"	内部工具 / 临时支撑系统

盘点下来发现，它们的技术栈、UI 库、请求库、表单方案几乎都不同，一份「放之四海皆准」的规范根本不存在。这就决定了 Skill 不能是一篇大而全的文档，而得先解决「AI 现在在哪个项目里」。

于是我反着问自己一个问题：

> 💬 ***一个前端老师傅接手一个陌生仓库时，脑子里在跑哪些判断？***

把这些判断一条条挖出来、写成 AI 能照做的指令，就是这个 Skill 的由来。落到结构上，大致是这么几层：

- **先认项目，再谈规范：**一个经验老到的前端第一反应是"这是哪个项目、什么栈"。我把这步固化成一套可执行的识别顺序，先查 `git remote` 反查仓库映射表，命中不了再读 `package.json` 的依赖信号，还不行就 `search_codebase` 兜底，实在拿不准就问人、绝不臆测。AI 由此能自动锁定该用哪套技术栈，而不是默认 `Next.js` + `Tailwind`。
- 每条**红线**都带上"为什么"：***禁 any、禁裸 fetch、禁 Tailwind、禁硬编码颜色、装包必须走 tnpm……***这些约束本身好写，难的是让 AI 不过度泛化、也不偷偷绕过。所以每条都补一句 why（比如"禁裸 fetch——拦截器/错误码/鉴权都在统一封装里"），AI 在解释和执行时会复述这层理由，约束因此既被遵守、又顺带提醒了使用者。
- **给标准答案，而不是讲道理：**把组件三段式、Service + Hook 双文件、列表页……些"团队公认的写法"做成最小可运行样板，AI 看到样板就照着改，比读十条规则都管用。

技术选型矩阵

- **不重复造能力，只做编排：**设计规范（稳定性 Token）、国际化（an-i18n-setup）、各项目专属写法（goc-frontend / x-ops 子 Skill），都已有各自的 owner 和产物。主 Skill 不把它们抄一遍，而是用引用的方式挂载进来，命中场景时再按需加载。

结合以上内容的梳理，最终沉淀为了前端 Skill：**`an-frontend-skill`**，并自然收敛出一套**五维结构**——从真实项目里「长」出来的。

04
an-frontend-skill 的五维结构



为什么是这样的五维？

AI Coding 的场景**不只是新项目，更多是在既有项目上的迭代**——技术债多、历史写法并存。我们要解决的核心问题是：让 AI 能识别项目现状、产生稳定代码、可控地升级重构。五维分工如下：

- When AI 什么时候该加载规范；
- **What** 在我们这个语境下该选什么；
- **Don't / Why** 绝对不能碰什么；
- **How** 标准答案长什么样；
- **Map** 团队设计 / 国际化等周边能力怎么挂载进来。

先看一眼 Skill 的内容目录

五维不是抽象概念，是实实在在的一组文件。`an-frontend-skill` 的目录大致如下：主入口 `SKILL.md` 负责「触发 + 路由 + 概要约束」，重内容拆到 `reference/` 下按需加载，特定项目的写法则以子 Skill 形式嵌入：

```
an-frontend-skill/
├── SKILL.md                    # 主入口：触发场景(When) + 项目
```

技术选型矩阵

```
└── reference/
    ├── aicoding-rules.md          # Don't/Why：完整硬性约束（MUST / MUST
    ├── tech-selection.md          # What：技术选型矩阵 + 内部优先/业界次选决
    ├── component-templates.md     # How：标准样板（组件三段式 / Service+Ho
    ├── coding-standards.md        # 代码规范详解（命名 / 目录 / 注释 / Git
    ├── yunan-xops-design.md       # Map：稳定性设计规范（Token / 排版 / 图
    ├── domain-repository-map.json# 路由事实源：仓库 → 项目类型 的反查表
    ├── glossary.md                # 术语表
    ├── contacts.md                # 问题对接路由（各能力 owner）
    └── skills/                    # 嵌入的项目专属子 Skill
        ├── goc-frontend/          #   经典稳态系（Ice.js + Fusion + React
        └── x-ops/                 #   稳定性标准系（@alife/x-ops-*，含完整
```

下面按五维逐一拆开讲。

维度 1：触发场景（When）

很多人写 Skill 第一行就是「# 前端开发规范」加一堆能力清单，AI 不知道什么时候该加载。我们把触发场景放最顶部，写得非常具体：

```
## 触发场景
**只要对话涉及 React/TSX 代码的生成、修改、重构、评审，即使用户未显式提及本 Skill
具体包括：
- 新建 React 项目脚手架
- 新增组件、Hook、页面
- 重构既有前端模块
- 接入团队内部组件库 / 设计系统
- 选型决策（"用什么状态管理"/"表单库选哪个"）
- 后端同学做端到端交付时的前端部分
- 特定项目（goc-frontend / x-ops / ...）的任何前端改动
```

维度 2：技术选型矩阵（What）

团队需要的是「闭眼选什么」。

我们的矩阵分两层：

第一层：项目类型路由（解决既有项目的识别问题）

项目	锁定技术栈	理由
历史项目 A	Ice.js 2.x + Fusion + Formily	历史决策，不动
历史项目 B	团队私有 Pro 组件库	强绑定中后台体系

技术选型矩阵

新项目	React 18 + TS + Vite + XOps + ahooks	本节真正的"选型"

第二层：新项目场景表

场景	首选	不推荐
UI 组件库	Ant Design	Tailwind（绕过 Token）
表单	Ant Design Form	Formily（社区萎缩）
状态管理	ahooks / Zustand	Ice Store（停止维护）
数据请求	内部请求库（magic-request）	SWR（与团队栈重复）
图表	AntV (G2/G6)	ECharts（包比较大）

维度 3：硬性约束（Don't / Why）

这是最有「团队味道」的一维。我们写了 8 条硬约束，每条都附 Why：

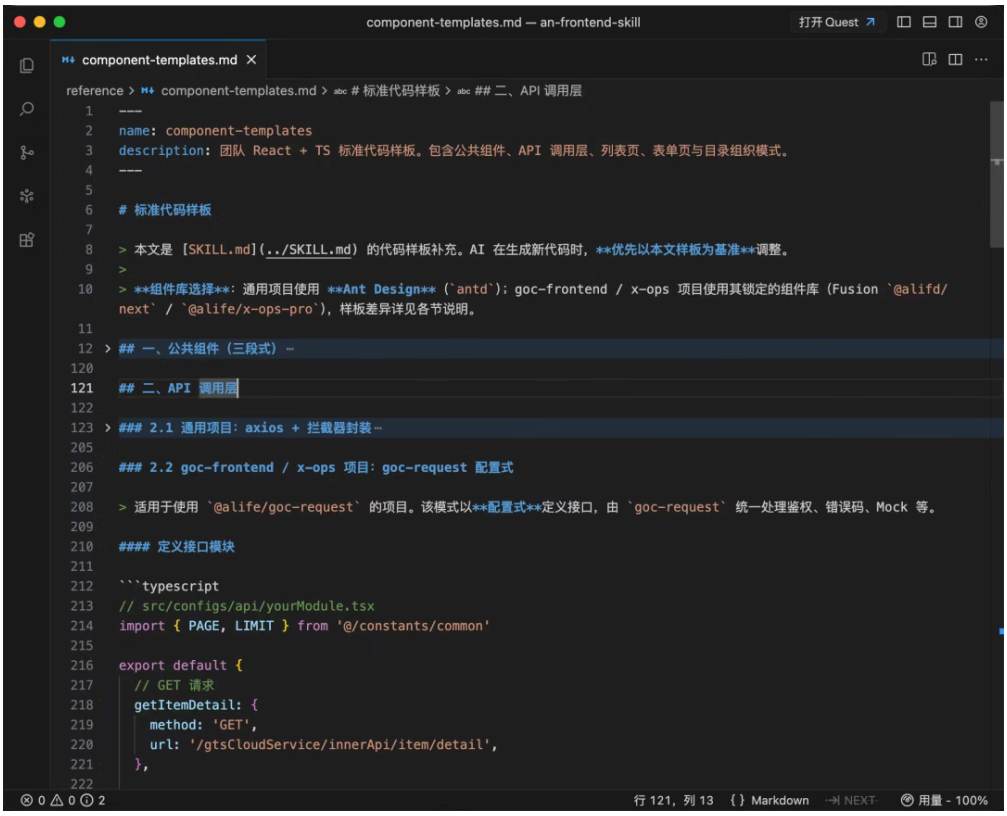
```
## 硬性约束（违反将被拒绝）
1.**禁用 any**：用 unknown + 类型守卫。Why：any 是 TS 的逃生舱，团队规约里逃生舱
2.**禁用 Tailwind**：统一 CSS Modules + Design Token。Why：原子类与 Token 体
3.**禁用第三方 iconfont / 散落 SVG**：统一使用团队私有 icon 包。Why：图标是品牌资
4.**Hook 必须在顶层调用**：禁条件 / 循环内调用。Why：React 规则，违反必出 bug。
5.**组件单一职责，单文件 < 200 行**：超出必须拆。Why：可读性 & 可测试性。
6.**API 调用必须走统一封装**，禁裸 fetch。Why：错误处理、loading、鉴权、埋点都在拦
7.**禁手写 useState + useEffect 模拟数据请求**：用 useRequest。Why：反模式，退
8.**禁硬编码颜色 / 间距 / 圆角**：必须用 Token。Why：Design System 存在的全部意
```

维度 4：标准样板（How）

如果只能给 Skill 留一个维度，我会留这个。**Skill 真正的杠杆是「标准答案」，AI 一旦看到一份好样板，就会自动用它做参照。**

我们提供了 4 套样板，每套都是「最小可运行 + 团队约定写法」：

- **公共组件三段式**：`index.tsx` + `index.module.css` + `types.ts` + 桶导出；
- **API 调用层**：双轨——通用项目用 axios + 拦截器；锁定栈项目用团队私有的配置式请求库；
- **列表页**：`useRequest + Table + columns prop` 的完整骨架；
- **表单页**：`Form + Form.Item + name + rules + onFinish` 的标准写法。



维度 5：跨能力集成（Map：设计 + 国际化）

前 4 维解决「代码本身怎么写对」，第 5 维解决「代码之外的团队能力怎么自动接入」。目前挂载了两类已有的团队产物：**设计语言**和**国际化方案**，Skill 的作用是让 AI 可消费，不是重写一份。

- ① **设计语言映射（稳定性设计规范）**：Design Token（颜色 / 间距 / 圆角 / 字号 / 阴影）+ 设计稿元素到 Ant Design / 团队私有组件的映射 + 图标库用法。由设计师同学的设计 Skill 集成进来——**前端 Skill 不再独立维护 Token，而是引用设计同学的产物**，权责清晰、单点更新。
- ② **国际化方案（`an-i18n-setup`）**：基于集团 `must` 国际化框架，在一国一云项目中已深度使用，覆盖文案抽取、key 生成、多语言文件组织、运行时切换降级。主 Skill 在涉及 i18n 场景时自动联动。

```
### 设计语言（必读）
> 涉及颜色 / 间距 / 圆角 / 阴影 / 图标 / 组件样式时，**必须**先加载：
> @skill: yunan-design-system
> 不要自己取色、不要硬编码 Token。
### Design Token 速查
| Token | 值 | 适用场景 |
|---|---|---|
| --color-primary |
#1057D9
 | 主色，按钮 / 链接 / 强调 |
| --color-warning |
#FA8C16
```

技术选型矩阵

```
 | 警告状态 |
| --radius-base | 4px | 普通容器 |
| --radius-card | 8px | 卡片 |
| --spacing-md | 16px | 常规间距 |
...
### 国际化（按需加载）
> 涉及多语言文案、key 生成、`<FormattedMessage />` 时，**必须**先加载：
> @skill: an-i18n-setup
> 默认所有用户可见文案走 i18n，硬编码中文需在 PR 里写明理由。
### 调用顺序
1. 主 Skill（an-frontend）→ 解析项目类型 + 注入硬约束
2. 命中设计场景 → 加载 yunan-design-system
3. 命中 i18n 场景 → 加载 an-i18n-setup
4. AI 生成代码 → 三方规范同时在场
```

05
实践落地案例



案例 1：Status 云产品健康看板——D2C/R2C 能力实战

场景：Status 云产品健康看板的开发，借助 **Aone Super** 官方平台的 D2C、R2C 能力实现，目前已上线。

项目规模：全新平台，1 个页面 / 10 个组件 / 9 个核心接口。

指标	数据
研发周期	3 周
AI 代码采纳率	80%
问题	1. D2C 还原效果一般，需要严格控制选区大小，出码后需人工反复微

技术选型矩阵

	2. 功能和交互效果实现依赖多轮对话，上下文长容易导致链路中断 3. 仅支持 `Claude-4.6-Sonnet` 模型（当前已支持 `Claude-Opus-4.7`）

【核心价值】 首次尝试 D2C/R2C 的可行性验证，为后面的 Skill 设计才提供了真实的踩坑依据。

案例 2：CFD 演练平台——老旧项目升级改造

场景：故障演练平台（CFD）从旧技术栈升级到新规范体系。去年团队下同类型其他业务平台的升级通常需要 **2-3 周**，加载 Skill 后 AI 自动遵守目标技术栈，不再需要人工逐文件对齐，**整体改造只用了 3 天。**

> **项目规模：**约 23 个页面模块 / 80+ 个组件（17 个公共组件 + 55 个页面级组件）/ 30 个 API 接口模块。
> 旧栈 Umi 3 + Ant Design Pro → 目标栈 Umi 4 + xops-design + ProComponents

指标	传统手工升级	AI + Skill 升级
改造周期	1-2 周	**3 天**
手动对齐规范的工作量	约 60% 时间花在样式/组件替换	**接近 0**
改造后代码一致性	参差不齐，需多轮 review	**首次产出即符合规范，局部微调**
状态	完成技术栈 + 组件库升级改造，具体模块页面样式持续优化中。	

【核心价值】Skill 约束下 AI 自动遵守目标技术栈，省掉了最耗时的逐文件手工对齐

案例 3：AIOps 项目——设计同学直接 AI Coding，原型即代码

场景：让设计同学绕过**"~~出设计稿 → 交付前端 → 前端还原 → 设计走查~~"**链路，直接 **AI Coding → 可交互原型。**

前提：

a. 前端建好 git 项目代码库 + 内置前端&设计 Skill 约束；
b. 设计同学能够上手 Qoder + Git，了解前端工作流（分支管理、O2 部署等）

效果：

1. 设计同学直接通过 AI Coding 实现可用于评审的**可交互原型，并可一句话实现 O2 部署**（项目中内置部署相关的 skill）
2. 评审反馈可直接在代码上调整，并支持**版本管理和切换**（可实时动态切换 cdn 资源版本号）
3. 交付给前端的项目代码，**可用率达到 70-80%**（符合前端 + 设计规范的合格代码）
4. 原型的快速迭代和评审，支撑了项目**双周迭代**的节奏目标（Scrum 模式推进的敏捷迭代）

> **项目规模：**约 3 个页面 / 24+ 个组件 / 8 个核心接口，涉及 SSE、动态数据渲染、钉钉卡片交互

技术选型矩阵

指标	传统协作链路	Skill + 设计直接 AI Coding
设计产出	静态设计稿 + 标注	可运行前端代码
协作环节	设计出稿 → 评审 → 前端还原 → 联调	**设计直接产出代码 → 评审 → 微调交付**
代码可复用率	< 30%（前端基本重写）	**70–80%（直接移植 + 适配）**
前端接入耗时	2–3 天	**半天**
信息传递损耗	高（设计标注→还原→走查多次转译）	**极低（设计意图直接固化为代码）**

【核心价值】 避免多环节 AI Coding 的代码浪费——**第一步产出的代码就能最后用在生产环境**，"原型即代码"从理念变成现实。

案例 4：一国一云国际化——Skill 驱动的批量改造

场景：一国一云需求要求多个平台支持中英文多语言。去年我们在故障平台、变更平台用内部**美杜莎**方案做过一轮国际化，虽然文案抽取可通过 must extract 批量完成，但模板字符串、Formily Schema 表达式、ProTable locale 注入等仍需人工逐文件处理，两个平台前后投入约 2 个月。这次重新推进其他业务平台合规化改造时，我们把整套国际化流程沉淀成了 `an-i18n-setup` Skill，让 AI 按流水线执行——**三个平台 2 周内全部完成改造。**

Skill 做了什么：一条 6 步自动化流水线 + 5 个配套脚本。AI 加载 Skill 后只需按顺序执行，且每一步都是幂等的——已处理的文件自动跳过，可以反复执行不出错。AI 只需要在极少数边缘场景（跨行模板、复杂嵌套）手动微调。

```
# 1. 自动抽取中文文案并包裹 $i18n.get()
echo "Y" | must extract
# 2. 处理 must extract 无法覆盖的模板字符串
node scripts/fix-template-literals.js
# 3. 处理行号偏移导致跳过的条目（模糊匹配兜底）
node scripts/fix-skipped-templates.js
# 4. 处理 Formily Schema 中 {{}} 表达式的中文
node scripts/transform-formily-i18n.js
# 5. 为所有 ProTable 批量注入 locale 配置
node scripts/add-protable-locale.js
# 6. 基于领域词典自动翻译 en-US.json
node scripts/translate-en-us.js
```

项目工作规模

技术选型矩阵

故障管理（GOC）

国际化需求工作规模总览

维度	数据
核心改造周期	2026-05-12 ~ 2026-05-26（15 天）
总提交数	12 次
变更文件数	665 个
代码变更行数	+52,846 / -8,031
净增行数	+44,815 行
i18n 词条总数	6,074 条

变更管控（CM）

国际化需求工作规模总览

维度	数据
核心改造周期	2026-05-08 ~ 2026-05-28（约 3 周）
总提交数	110 次（有效提交 97 次）
变更文件数	717 个
代码变更行数	86,288 行（+61,855 / -24,433）
净增行数	+37,422 行
i18n 词条总数	zh-CN 5,517 条 / en-US 5,518 条

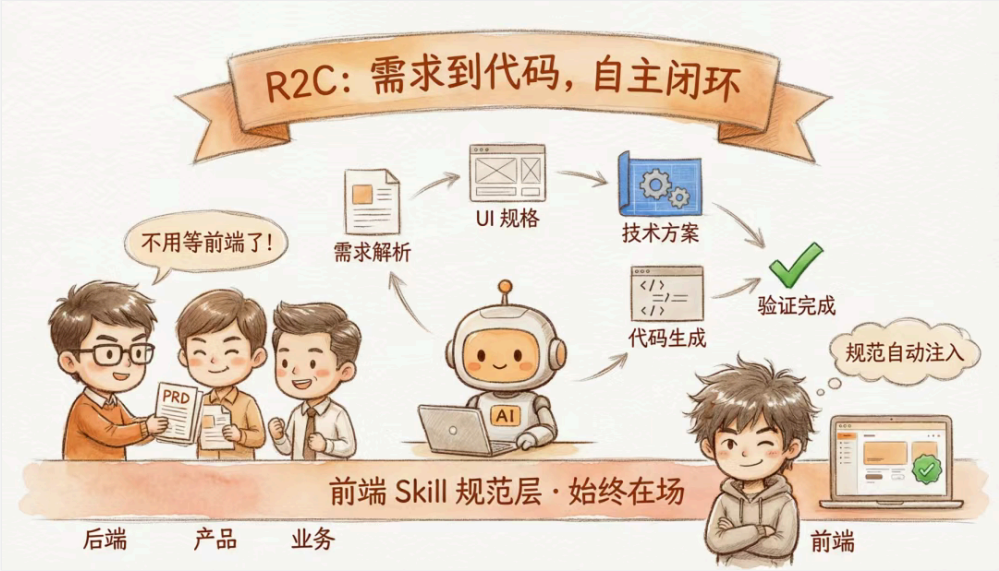
风险巡检（CIS）

国际化需求工作规模总览

维度	数据
核心改造周期	2026-05-08 ~ 2026-05-22（2 周）
总提交数	**89** 次
变更文件数	**173** 个
代码变更行数	**25,051**（+21,589 / -3,462）
净增行数	**+18,127** 行
i18n 词条总数	**2,366** 条（中英文）

【核心价值】 Skill 把国际化从一次性人力活变成可复用的自动化流水线

06
场景延伸：R2C 需求转代码



在 Skill 基座之上，我们进一步探索了 **R2C（Requirement-to-Code）**：拿到 PRD 后通过多个 Skill 串联，让低复杂度需求（管理后台增删改查类）由后端/产品/业务同学**自主闭环**，降低前端依赖。

用法很轻：在 IDE 里丢一个钉钉文档链接（可选附设计稿）+ 指定仓库路径，它就自动走完从需求分析到代码生成的整条链路。目前已沉淀为 `an-r2c` Skill，覆盖研发链路的完整流程：

阶段	内容	状态
PM 阶段	PRD 解析 → 用户确认 → `requirement.md`	已实现
UI 规格阶段	设计稿/D2C DSL 解析 → 用户确认 → `ui-spec.md`	已实现
技术方案阶段	仓库分析 + 范围确认 + 方案生成 → `tech-solution.md`	已实现
代码生成阶段	路由 / 接口 / UI / 逻辑，全自动串行	已实现
验证阶段	启动 dev server + 视觉校验	已实现
AI Code Review	AI 自动 Review 产出代码是否符合 Skill 约束	

技术选型矩阵

部署触发	自动触发构建和部署流程	规划中
线上验证	部署后自动化验证	规划中

三个让后端 / 产品也用得起来的关键设计：

- **流程拆成 10 个阶段文件**：每个阶段只管一件事（做什么、不做什么、出错怎么兜底），对使用者透明、对维护者可单点修复；也规避了"规则堆在一个大文件里、超过 4000 字后 AI 遵循度下降"的坑。
- **依赖一键集成**：把 6 个子 Skill（钉钉文档、设计规范、组件库、代码规范、脚手架、接口联调）随主 Skill 打包分发，安装成本从"装 6 个 + 配 MCP"降到"装一个 + 首次授权一次"。
- **写码前先出结构化 Spec**：先给出需求摘要 / 页面 / 接口 / 交互 / 改动文件 / 验收清单，确认后才生成代码。实测约 30% 的场景会在这一步就暴露需求与设计稿的不一致——否则要拖到 UI 走查才被发现。

前三阶段是 **「输出快照 → 用户确认 → 落盘 + git commit」** 循环；技术方案确认后进入全自动生成。整个流程里，前端 Skill 作为底层约束始终在场，产出的代码在 code review 时与前端手写的产物基本无差别。

落地后的协作变化：一个标准 CRUD 需求，从需求文档到浏览器跑起来约 20 分钟，对比过去「后端试写前端 → 前端帮忙 review 改完」的 2–3 天大幅压缩。角色也随之重构：后端开始端到端交付、产品自己开发需求、前端从「写每个页面」转向「review AI 产出 + 攻坚复杂场景」。

全链路闭环还需要后端能力配合

端到端闭环要真正打通，仅靠前端 Skill 不够，同时也需要后端协同：

- **接口文档按照 OpenAPI 要求规范化，AI 才能自动生成类型和请求代码；**
- **状态码 / 错误码 / 提示信息统一，AI 才能做通用错误处理；**
- **跨端知识库共建。**

07
后续规划

规划内容	阶段
团队跨端知识库建设（持续推进中）	进行中
集成现有的构建工具和部署流程，实现 AI 研发流水线后续环节（CR、CI/CD）	进行中
推动后端接口文档 OpenAPI 的规范化和状态码的统一化	计划中
更多项目验证 + 落地数据回收	计划中

技术选型矩阵

08
尚未解决的难题：AI Coding 提效量化

效率提升是直观可感的，老平台升级改造从 1–2 周压到 3 天、新项目原型代码可用率从 < 30% 提到 70–80%。但坦诚说一个挑战：**AI Coding 提效的量化目前没有统一标准：**

- 效率用什么口径?
- 代码采纳率怎么定义（改 3 行算不算）？
- 同一需求不同人的差异归因给工具还是人?
- Skill 自身的效果除了下载量和用户反馈也很难度量。

我们可以考虑的做法是先用**「可观测对比」代替精确量化**，用 3 个粗粒度口径自我观测：

1. **改造前后人天对比**：同类规模下无 Skill vs 有 Skill 的人天差（如案例 2 的 1–2 周 → 3 天）；
2. **首次产出 review 通过率**：AI 第一次产出代码需要打回返工的比例（目标从 60%+ 降到 < 20%）；
3. **Skill 使用侧信号**：下载量、收藏量、主动反馈数，衡量「是否真的被用起来」。

这套口径不完美，衡量不了「AI 帮我想出了我自己想不到的方案」这种隐性价值，但**先有粗的、可观测的口径，比等一个完美指标更重要。**

09
思考：能力沉淀 > 个人提效

个人写得快不是竞争力，把「怎么写得对」沉淀成可复用的能力资产，让所有人都写得对，才是。



从个人能力到组织能力：Skill 是规模化杠杆

技术选型矩阵

很多人对 AI Coding 提效的理解，停留在「我自己变强」这一层。用上最先进的工具、调动最强的模型，把活干得又快又好。这当然是好事，但它解决的是个人天花板，而**团队的产出从来不取决于最强的那个人，而取决于最弱的那一环。**

因为现实是参差的：

- **部分生态同学用的还是 VSCode + AoneAgent 或通义零码这类轻量级工具**，模型能力本身就弱一档，加上 token 额度有限，经常写到一半就得省着用——同样一个需求，持有先进生产力工具的同学可能一轮对话就能出初稿，工具弱的同学得拆成好几轮、还得手动补 AI 没写完的部分。
- **新人入职或者跨项目支援**，对项目历史背景一无所知，AI 也一样不知道。没有 Skill 约束的话，AI 会按自己的默认偏好写代码，新人还得自己判断哪些能用哪些不能用，结果就是写了一堆**"看着对但实际踩坑"**的代码，reviewer 返工量比不用 AI 还大。

所以如果只有少数人能写出高可用的代码，团队整体的代码**采纳率、返工率、token浪费**一个都不会变好。真正的杠杆不是让能力出众的人更快，而是把所有人的底线一起抬上来；哪怕用着普通的工具、有限的额度，也能稳定产出符合规范、可被接手的代码。

Skill 就是这个杠杆。它把「资深前端脑子里的判断」固化成一段 AI 每次都会自动加载的上下文，边际成本几乎为零：写一次，团队里每个人、每次对话都在复用。比起在群里喊一句「别用 Tailwind」，只有看到的人、记得住的人、当下在写这块的人才会执行；写成 Skill 后，这个决策会在每一次 AI 生成代码的现场自动生效，不依赖任何人的记性和自觉。

所以，也非常建议大家在做个人提效的时候，多问一句：

> 💬 ***这个流程能不能复用？ 换个人走一遍，能不能拿到同样的结果？还有没有更简单的方式？***

如果答案是肯定的，那它就该被沉淀成 Skill，从**「我会」变成「AI 帮所有人都会」。**

目前 `an-frontend-skill` 已推给团队前端以及 PD 同学在多个产品项目上使用，接下来会继续在后端同学端到端的项目中发力：前端 skill 已经是**面向多角色的团队基础设施**。

给其他团队的建议：从真实痛点开始

这套思路对其他团队同样有参考价值：方法论可以照搬（五维结构、规范前置、用 Skill 替代平台）；大部分能力可以复用（硬约束、样板、设计映射）；真正团队特有的部分不到 30%（主要是历史栈锁定和内部组件映射）。

AI 时代真正稀缺的能力是**发现痛点的洞察力**，技术实现门槛已经很低，但能不能**从日常工作流里捕捉到那些反复消耗时间、又没人系统解决的环节**，才是关键。建议**从自己的研发或业务流程出发，锁定一个反复出现的痛点，用 AI 解决它，然后把方案抽象成 Skill**。两个我们做过的例子

- **配图风格不统一：**把一套稳定出图的 prompt 模板沉淀成了 Skill，任何人任何工具都能生成统一风格的配图（**Q萌手绘**）。

技术选型矩阵

- **画架构图太耗时：**基于 Drawio 抽象出 AI 辅助画图 Skill，几句话就能生成结构清晰的高质量图表（**ai-drawio**）。

这些能力都是从**真实场景**里长出来的。先解决自己的问题，再把解法泛化成可复用的工具——这个路径对团队里每个人都适用。



10
回顾与展望

这半年走下来，核心认知转变就一个：**规范落地的切入点是 AI 的上下文**。课程、文档、平台解决的都是**「人知不知道」**的问题，但 AI Coding 场景下真正决定产出质量的是**「AI 生成那一刻的上下文里有没有规范」**。Skill 本质上是把规范从被动参考变成主动注入。

接下来的演进方向分两条线：

1. **横向扩角色**——后端接口契约（OpenAPI spec → Skill）、测试用例生成约束、PD 原型工程化规范，**让 Skill 进一步从前端专属变成多角色共享的研发基础设施**；
2. **纵向延链路**——当前只覆盖了代码生成阶段，下一步计划接入 code review 自动化、CI 前置校验、部署后巡检，形成"生成 → 审查 → 验证"的完整 Skill 链。

最终实现 AI Coding 的理想状态：

团队研发流程的每个质量关口都有 Skill 约束在场，AI 在整条链路上都带着团队经验运行。

技术选型矩阵



欢迎留言一起参与讨论~

阅读 1.2万　修改于2026年7月30日

留言 18

写留言

木鱼　北京　8月2日　4

现在的AI已经会自己读取项目并按现有规范开发了，无需过多约束

赢　河南　8月9日

技术选型矩阵

是的啊，AI太强了现在



C 江苏 8月13日 2

一直不理解为什么要前端后端都要写，难道不是前端+Ai，后端+AI，各自在各自专业的位置coding不是更好吗，如果说对接浪费时间，但是AI提效节约的时间完全覆盖了，个人觉得应该是想办法通过AI提效对接效率就行，比如通过AI约束输出的接口啥的必须是什么样的，提交的数据格式必须是什么样的



药学人玩AI 上海 8月1日 2

写代码的人肯定要为自己写的代码负责，你不能后端写前端代码有问题就让前端处理，这样代码职责混乱，谁都不会认真写的



有青 浙江 8月2日

理解你的顾虑，不过这就是我们做 skill 的一个原因——让 AI 约束后端和其他角色产出的前端代码，职责没变乱，只是前端从"逐行还原"转成"review AI 产出 + 攻坚复杂场景"。当然我们也没法要求非前端角色能写出100%符合要求的页面代码，skill也需要持续更新迭代的。



不喝啤酒 江苏 7月31日

这个前端skill开源了吗?



有青 浙江 8月2日 1

暂时没开源哈。skill 里绑了一些我们内部的技术栈、私有组件库和业务经验，不太适合直接复用。不过文章里思路和目录结构都写得比较细了，照着这套多维结构让 AI 帮你整一份符合自己项目和团队现状的skill或者plugin，效果应该是更好



简简单单 广西 7月30日 1

深度好文，很实在很落地

首评



The Fall 安徽 8月16日

我们是单独建了一个 ai kit 放前后端的共用的知识库和 skills 现在需求也在啊 kit 按照项目场景进行划分 并自动区分是通用的知识沉淀还是项目的知识沉淀 同步在各个目录中 agents 进行路由



John 江苏 8月9日

随着模型能力日益提升，公用约束不能太多，需要聚焦到自身业务约束



。 浙江 8月5日

感谢分享，想请问下后端服务是在哪个阶段对接的



天护子女 江苏 8月4日

加油加油



烟雨南下 浙江 8月1日

感谢分享，转发技术研究



Christopher 广东 7月31日

开源不

技术选型矩阵

张湘南 湖南 7月31日

技能能分享不

产品经理的AI... 上海 7月31日

不懂就问，那为啥不用项目级的claude.md+rules呢?

有青☼ 浙江 8月2日

你好，理解你的疑问，其实这俩是不同场景的工具，不是替代关系。
claude.md + rules 比较适合特定的仓库：这个项目特有的约束、目录结构、命名习惯，跟着 repo 走、长期维护，团队固定、就在这一个库里干活时最顺手。
skill 适合"跨仓库、跨角色复用"：一套规范想发给多个个技术栈不一样的项目、还想给后端/PD/设计一起用，这时候你需要的是能分发、能被自动识别加载、能单点更新全员升级的东西——rules 绑死在单个仓库里做不到这些。
所以选择上，可以遵循这个原则：团队通用的沉进 skill 跨项目分发，各仓库自己的个性化约束可以继续放项目级 .md + rules

蜻蜓飞呀飞–... 湖南 7月31日

开源了吗

技术选型矩阵